Полное руководство по архитектуре, настройке и работе с Coder и Docker

Раздел 1. Архитектурный облик Coder и Docker

1. Где работает сам Coder?

Coder работает внутри Docker-контейнера.

Ваш стек Coder (сервер Coder + база данных PostgreSQL) — это такие же контейнеры, запущенные на вашей хост-машине через `docker compose`.

- **Суть:** Coder не устанавливается напрямую в операционную систему Windows. Он заперт внутри своего собственного контейнера `ghcr.io/coder/coder`.

2. Coder — это управляющий элемент?

Совершенно верно.

Coder — это центральный «диспетчер» или оркестратор:

- Он хранит настройки пользователей и шаблоны (в PostgreSQL).
- Он предоставляет веб-интерфейс (`http://localhost:3000`).
- Он отдает команды Docker на создание, остановку или удаление рабочих пространств (Workspaces).

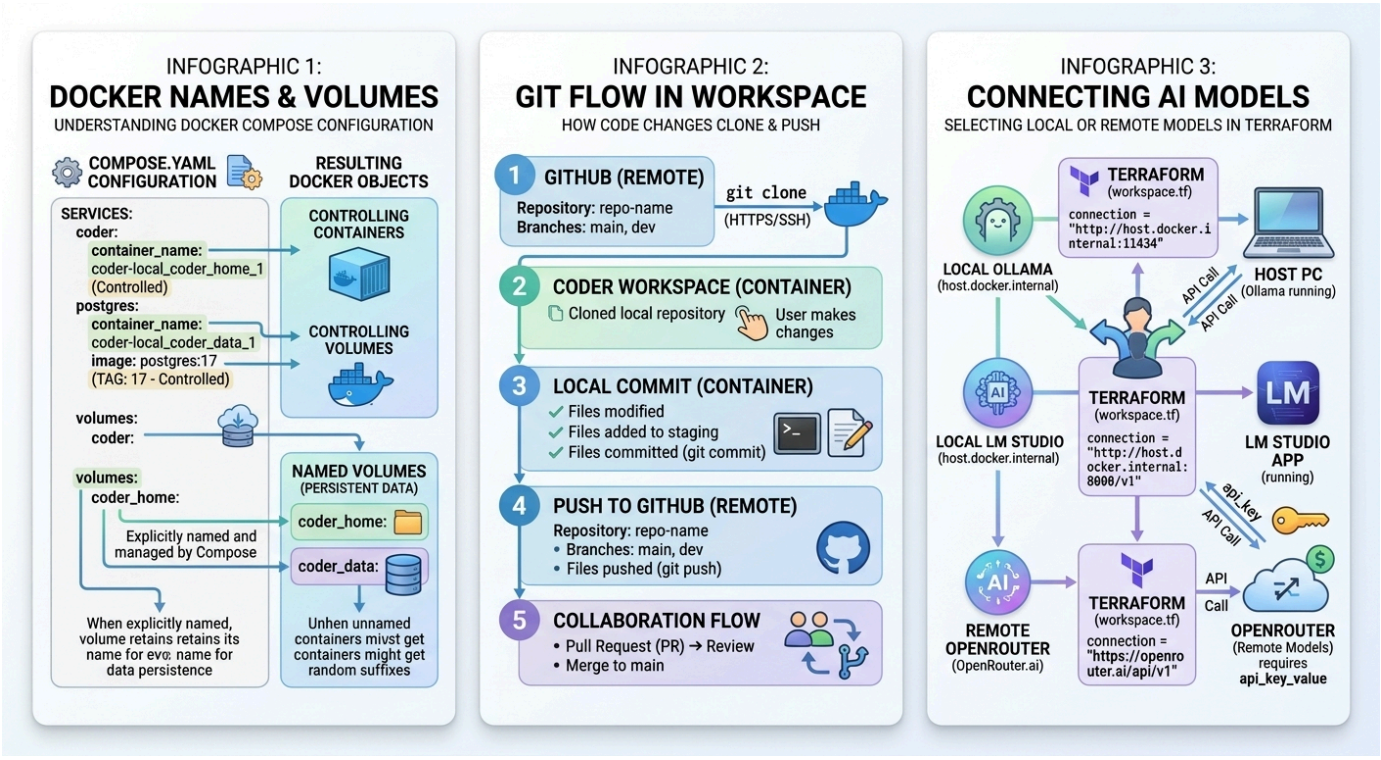
3. Что такое Рабочее пространство (Workspace)?

Рабочее пространство — это отдельный изолированный Docker-контейнер, созданный Coder'ом по вашему требованию.

- Вы нажимаете кнопку `+ Create Workspace` в веб-панели Coder.
- Coder берет шаблон (Terraform script) и командует вашему Docker Desktop: *«Запусти новый контейнер из образа Ubuntu»*.
- В результате создается независимый контейнер (например, `coder-sxidsvit-coder-test`), внутри которого установлена ОС Linux и инструменты разработки.

🎨 Визуальный блок 1: Основные концепции (Инфографика 1-3)

На представленной ниже инфографике отображены три ключевых архитектурных блока системы:



Подробный разбор панелей 1–3:

Панель 1. DOCKER NAMES & VOLUMES (Имена Docker и Тома)

Объясняет связку между файлом конфигурации `compose.yaml` и создаваемыми объектами Docker:

- **Управляемые контейнеры:** Прямые стрелки от `container_name:` задают понятные имена контейнерам (`coder-local`, `coder-postgres`).
- **Постоянные тома (Named Volumes):** Явное объявление секций `volumes:` создает независимые хранилища (`coder_home` и `coder_data`), сохраняющие данные даже при полном пересоздании контейнеров.

Панель 2. GIT FLOW IN WORKSPACE (Цикл работы с Git)

Визуализирует 5 шагов работы с кодом:

1. **GitHub (Remote):** Клонирование репозитория по SSH/HTTPS в контейнер.
2. **Coder Workspace:** Внесение изменений в файлы внутри воркспейса.
3. **Local Commit:** Индексация (`git add`) и фиксация (`git commit`) — изменения остаются локально в контейнере.
4. **Push to GitHub:** Отправка коммитов (`git push`) в удаленный репозиторий.
5. **Collaboration Flow:** Создание Pull Request и слияние изменений с веткой `main`.

Панель 3. CONNECTING AI MODELS (Подключение AI-моделей)

Показывает 3 маршрута подключения AI через файл `workspace.tf`:

- **Local Ollama:** Запросы через `host.docker.internal:11434` к Ollama на Windows.
- **Local LM Studio:** Запросы через `host.docker.internal:8000/v1` к LM Studio на Windows.
- **Remote OpenRouter:** Внешние вызовы API к `openrouter.ai/api/v1` с передачей API-ключа.

Раздел 2. Подробно про тома `coder_home` и `coder_data`

Для работы Coder используются два основных именованных Docker-тома (Named Volumes). Тома создаются Docker Desktop на вашем компьютере и сохраняются даже после полного выключения ПК.

1. Зачем нужен том `coder_data`?

- **Что хранит:** Базу данных **PostgreSQL** (учетные записи пользователей, роли, пароли, загруженные шаблоны Terraform, историю вызовов и логи воркспейсов).
- **Где используется:** Подключается к контейнеру базы данных по пути `/var/lib/postgresql/data`.
- **Зачем нужен:** Без этого тома при остановке контейнера PostgreSQL все ваши аккаунты, настройки администратора и шаблоны Coder полностью стирались бы.

2. Зачем нужен том `coder_home`?

- **Что хранит:** Домашнюю папку управляющего сервера Coder (`/home/coder`), включая системные ключи шифрования, локальный кэш шаблонов и конфигурационные файлы самого сервера Coder.
- **Где используется:** Подключается к контейнеру Coder Server по пути `/home/coder`.
- **Зачем нужен:** Обеспечивает стабильность работы сервера, сохраняет сессии и внутренние служебные данные сервера между перезапусками.

3. Где физически хранятся эти тома на диске Windows?

Поскольку Docker Desktop в Windows работает через подсистемы **WSL 2** (Windows Subsystem for Linux), эти тома **не находятся** в обычной папке вроде `C:\Users\...`.

Физически они упакованы внутри виртуального диска WSL 2:

```
\\wsl$\docker-desktop-data\data\docker\volumes\
  ├── coder-local_coder_data\_data
  └── coder-local_coder_home\_data
```

4. Как посмотреть содержимое томов `coder_home` и `coder_data`?

Так как тома находятся внутри изолированной среды Docker/WSL2, просто так открыть их через двойной клик в «Проводнике» Windows не получится. Но есть **3 удобных способа** заглянуть внутрь:

Способ 1. Самый простой (через Docker Desktop GUI)

1. Откройте **Docker Desktop**.
2. В левом боковом меню перейдите во вкладку **Volumes**.
3. Найдите в списке нужный том (например, `coder-local_coder_home` или `coder-local_coder_data`).
4. Кликните по названию тома и перейдите во вкладку **Data**.
5. На этой вкладке отобразится встроенный проводник по файловой системе тома, где можно просматривать и скачивать файлы.

Способ 2. Через временный Docker-контейнер (в PowerShell)

Вы можете запустить мгновенную служебную команду Docker, которая подключит том и покажет список файлов внутри:

- **Посмотреть файлы `coder_home`:**

```
docker run --rm -it -v coder-local_coder_home:/volume alpine ls -la /volume
```

- **Посмотреть файлы `coder_data` (база данных Postgres):**

```
docker run --rm -it -v coder-local_coder_data:/volume alpine ls -la /volume
```

Способ 3. Через Проводник Windows (WSL 2 путь)

Вы можете открыть содержимое виртуального диска Docker прямо в стандартном «Проводнике» Windows:

1. Нажмите сочетание клавиш `Win + R`.
2. Введите следующий адрес и нажмите **Enter**:

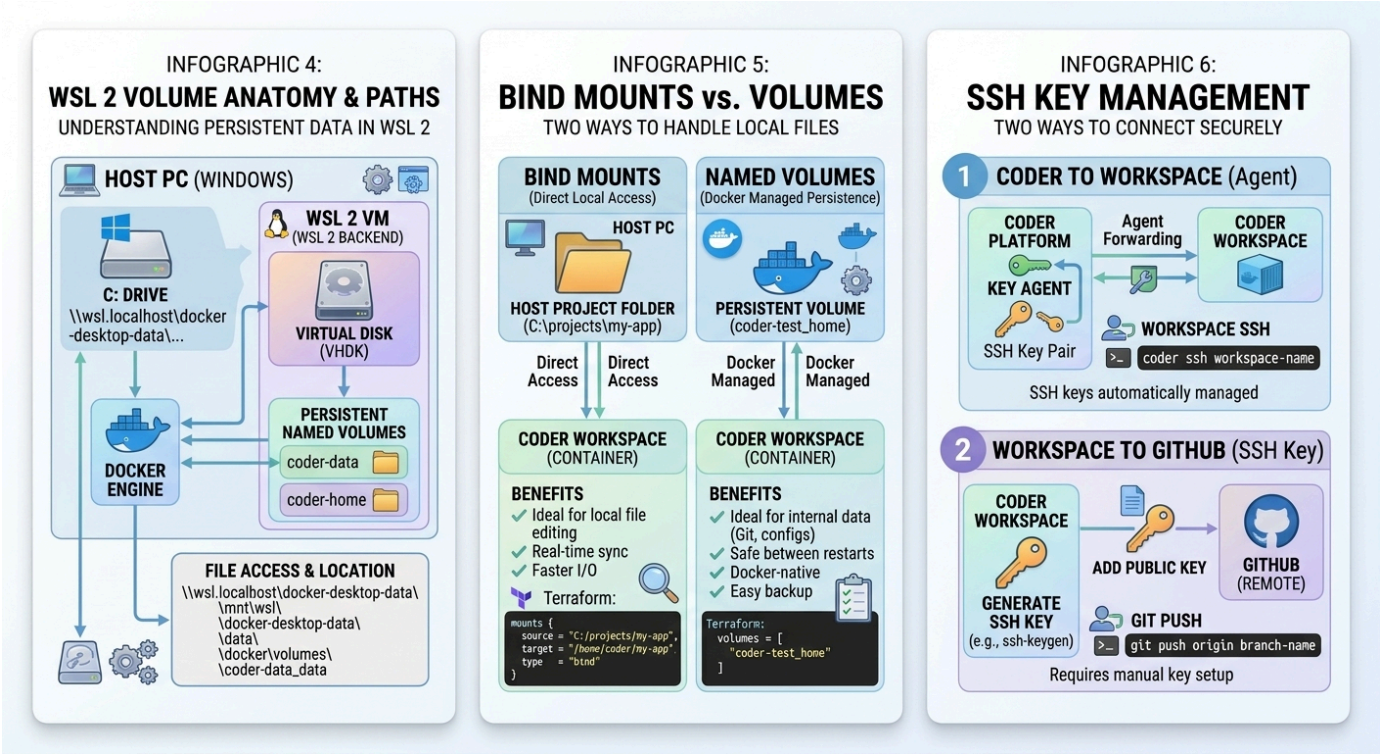
```
\\wsl$\docker-desktop-data\data\docker\volumes
```

(Или `\\wsl.localhost\docker-desktop-data\data\docker\volumes` в Windows 11)

3. Вы увидите папки томов, внутри которых в директории `_data` будут лежать физические файлы.

🎨 Визуальный блок 2: Продвинутые механизмы (Инфографика 4-6)

Вторая часть графических схем дополняет руководство техническими деталями хранения данных, примонтирования папок и SSH-авторизации:



Подробный разбор панелей 4–6:

Панель 4. WSL 2 VOLUME ANATOMY & PATHS (Анатомия томов WSL 2)

Детализирует физическое размещение данных в ОС Windows:

- **Host PC (Windows):** Диск `C:` и доступ через UNC-пути.
- **WSL 2 VM (Backend):** Изолированный виртуальный диск (`VHDK`), внутри которого Docker хранит тома `coder-data` и `coder-home`.
- **File Access & Location:** Указывает полный внутренний путь к данным томов: `\\wsl.localhost\docker-desktop-data\mnt\wsl\docker-desktop-data\data\docker\volumes\`.

Панель 5. BIND MOUNTS vs. VOLUMES (Сравнение типов монтирования)

Сравнивает два варианта работы с файлами:

- **Bind Mounts:** Прямой проброс локальной папки Windows (например, `C:\projects\my-app`) в контейнер. Мгновенная двухсторонняя синхронизация, идеально для работы с файлами проектов.
- **Named Volumes:** Тома под управлением Docker Desktop (`coder-test_home`). Максимальная изолированность и безопасность при перезапусках.

Панель 6. SSH KEY MANAGEMENT (Управление SSH-ключами)

Разделяет два независимых контура аутентификации:

1. **Coder to Workspace (Автоматически):** Агент Coder и CLI прозрачно пробрасывают SSH-сертификаты при запуске команды `coder ssh workspace-name` или открытии VS Code.
2. **Workspace to GitHub (Вручную):** Для отправки кода на GitHub внутри контейнера генерируется публичный ключ, который добавляется в профиль GitHub.

Раздел 3. Как формируются и как управлять именами контейнеров, томов и образов

Чтобы в списке Docker Desktop не путаться в длинных или случайных названиях, важно понимать алгоритмы их генерации.

1. Как формируются имена томов (`coder-local_coder_home`)

Когда вы указываете в `compose.yaml` именованный том:

```
volumes:
  coder_home:
```

Docker Compose по умолчанию добавляет в начало **имя проекта (префикс)**, разделяя его нижним подчеркиванием:

> **Имя проекта** + `_` + **Имя тома в YAML** → `coder-local_coder_home`

Именем проекта по умолчанию становится **название папки**, в которой лежит файл `compose.yaml` (например, папка `coder-local`).

Как этим управлять:

- **Вариант А. Задать имя проекта явно в `compose.yaml`:** Добавьте строчку `name` в самое начало файла:

  ```
  name: mydev

  services:
    coder: ...
  volumes:
    coder_home:
  ```

  Результат: Том будет назван `mydev_coder_home`.

- **Вариант Б. Задать точное имя тома вручную (без автоматических префиксов):** Используйте свойство `name` внутри секции `volumes`:

  ```
  volumes:
    coder_home:
      name: my_custom_coder_home
  ```

Результат: В Docker Desktop том будет точно называться `my_custom_coder_home`.

---

2. Как формируются имена контейнеров

По умолчанию Docker Compose называет контейнеры по шаблону: `[ИмяПроекта]-[ИмяСервиса]-[Номер]`.

Как этим управлять:

Вы можете переопределить стандартное имя прямо в `compose.yaml` с помощью параметра `container_name`:

```
services:
  coder:
    image: ghcr.io/coder/coder:latest
    container_name: my-coder-server # Точное имя контейнера в Docker Desktop

  postgres:
    image: postgres:17
    container_name: my-coder-db # Точное имя контейнера в Docker Desktop
```

Имена контейнеров-воркспейсов (создаваемых Coder):

Для воркспейсов имена задаются не в `compose.yaml`, а в **шаблоне Terraform** (`main.tf`) с помощью строчки:

```
resource "docker_container" "workspace" {
  name = "coder-${data.coder_workspace.me.owner}-${data.coder_workspace.me.name}"
}
```

- `data.coder_workspace.me.owner` — имя вашего пользователя в Coder (например, `sxidsvit`).
- `data.coder_workspace.me.name` — имя воркспейса (например, `coder-test`).

Результат: Контейнер воркспейса получает имя `coder-sxidsvit-coder-test`. Вы можете изменить этот префикс в шаблоне Terraform на любой другой (например, `dev-workspace-${data.coder_workspace.me.name}`).

---

3. Как формируются имена образов (Images)

Имя образа состоит из трех основных частей:

> **[Реестр / Владелец] / [Название Образа] : [Тег / Версия]**

- `ghcr.io/coder/coder:latest`
  - `ghcr.io` — реестр (GitHub Container Registry).
  - `coder/coder` — владелец и имя приложения.
  - `:latest` — тег (версия).
- `postgres:17`
  - `postgres` — официальный образ из Docker Hub.
  - `:17` — версия PostgreSQL.
- `codercom/example-base:ubuntu`
  - `codercom/example-base` — репозиторий образов.
  - `:ubuntu` — версия операционной системы.

Как этим управлять:

Вы указываете нужный образ в параметре `image:` в `compose.yaml` или в Terraform-файле. Вы всегда можете заменить базовый образ на свой собственный или на другую фиксированную версию (например, `postgres:16-alpine` или `ubuntu:22.04`).

Раздел 4. Настройка и пошаговый запуск Docker Compose

Конфигурационный файл `compose.yaml`:

```
name: coder-local

services:
```

```
  coder:
    image: ghcr.io/coder/coder:latest
    container_name: coder-local
    environment:
      CODER_HTTP_ADDRESS: '0.0.0.0:3000'
      CODER_ACCESS_URL: 'http://localhost:3000'
      CODER_PG_CONNECTION_URL: 'postgres://coder:coderpass@postgres:5432/coder?sslmode=disable'
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
      - coder_home:/home/coder
    ports:
      - '3000:3000'
    depends_on:
      - postgres
    restart: unless-stopped

  postgres:
    image: postgres:17
    container_name: coder-postgres
    environment:
      POSTGRES_USER: coder
      POSTGRES_PASSWORD: coderpass
      POSTGRES_DB: coder
    volumes:
      - coder_data:/var/lib/postgresql/data
    restart: unless-stopped

volumes:
  coder_home:
    name: coder-local_coder_home
  coder_data:
    name: coder-local_coder_data
```

Последовательность первого запуска платформы:

1. Откройте терминал PowerShell в папке с вашим `compose.yaml`.
2. Запустите сборку и запуск контейнеров в фоновом режиме:

   ```
   docker compose up -d
   ```

3. Docker скачает необходимые образы, поднимет контейнеры `coder-local` и `coder-postgres`.
4. Перейдите в ваш веб-браузер на Windows и откройте адрес:

   ```
   http://localhost:3000
   ```

5. При первом входе создайте аккаунт администратора, выберите или загрузите шаблоны (Templates) и нажимайте `+ Create Workspace` для разворачивания рабочих песочниц.

Раздел 5. Восстановление рабочей среды после выключения ПК

Когда вы завершаете рабочий день и выключаете ноутбук, все данные сохраняются в томах `coder_home` и `coder_data`. Ничего не удаляется.

Порядок действий при включении компьютера (на следующий день):

1. **Включите ПК и дождитесь запуска Docker Desktop.** Убедитесь, что значок Docker горит зеленым в трее Windows.
2. **Откройте терминал PowerShell** и выполните команду для поднятия сервисов сервера:

   ```
   docker compose up -d
   ```

   (Если в `compose.yaml` прописано `restart: unless-stopped`, Docker Desktop часто сам автоматически восстанавливает работу сервисов при старте).
3. **Перейдите в веб-интерфейс Coder:** откройте `http://localhost:3000`.

4. **Запустите воркспейс:** Напротив вашего нужного воркспейса (например, `coder-test`) нажмите кнопку **Start** (если он остановлен), а затем кликните по **VS Code Desktop**.
5. Настоятельно рекомендуется запускать/останавливать именно *воркспейсы* через веб-панель Coder, а не удалять сами контейнеры вручную из Docker Desktop.

Раздел 6. Шпаргалка по командам Docker

Управление стеком Coder:

- **Запустить стек в фоновом режиме:**

```
docker compose up -d
```

- **Остановить стек (сохранив все данные в томах):**

```
docker compose stop
```

- **Остановить и полностью удалить контейнеры сервиса Coder:**

```
docker compose down
```

- **Посмотреть список только запущенных контейнеров:**

```
docker ps
```

Очистка системы:

- **Полное удаление неиспользуемых контейнеров, образов и томов:**

```
docker system prune -a --volumes
```

Раздел 7. Откуда берутся файлы и как работает Git/GitHub

Существует **два принципиально разных способа** работы с вашими проектами в VS Code:

Вариант 1. Работа с GitHub-репозиторием (Жизненный цикл изменений)

Когда вы подключаетесь к воркспейсу через VS Code и склонировали репозиторий (`git clone https://github.com/user/project.git`), все файлы сохраняются **локально внутри контейнера воркспейса**.

Что происходит, когда вы вносите изменения в код?

Любые изменения (редактирование файлов, создание новых скриптов) **не попадают автоматически на GitHub**. Они находятся только на локальном файловом диске вашего Linux-контейнера.

Пошаговый процесс передачи изменений обратно на GitHub:

1. **Изменения в контейнере (Unstaged Changes):** Вы редактируете код в VS Code. Файлы помечены как измененные только внутри контейнера.
2. **Фиксация коммита (Local Commit):** Вы делаете коммит через интерфейс VS Code (вкладка Source Control) или через терминал:

```
git add .
git commit -m "Добавил новую функцию"
```

   Результат: Коммит сохранен в локальной истории Git внутри контейнера. На GitHub его еще нет.
3. **Отправка изменений на GitHub (Git Push):** Чтобы передать коммиты на сервер GitHub, выполняется команда:

```
git push origin main
```

(или в новую ветку: `git push origin feature/new-ai-agent`).

4. **Что изменяется на GitHub?**
   - Если вы пушите в ветку **`main`** (и у вас есть права прямого доступа): Изменения сразу обновляют основной репозиторий на GitHub.
   - Если вы работаете в команде (или в ветке **`feature/...`**): Изменения создают новую ветку на GitHub. Затем на сайте GitHub вы можете нажать кнопку **Create Pull Request (PR)**, чтобы проверить код перед вливанием в `main`.

> **Главное правило:** Контейнер воркспейса работает как ваш обычный локальный компьютер. Никакие изменения не уходят на GitHub до тех пор, пока вы явно не вызовете команду `git push`.

Вариант 2. Работа с папкой прямо с вашего диска Windows (Локальный проект)

Вы **можете работать с проектом, лежащим на вашем диске C:**, но для этого этот путь нужно заранее прописать (примонтировать) в шаблоне Terraform.

В Terraform-шаблоне воркспейса прописывается блок `volumes`:

```
resource "docker_container" "workspace" {
  image = "codercom/example-base:ubuntu"
  name  = "coder-${data.coder_workspace.me.owner}-${data.coder_workspace.me.name}"

  # Монтируем папку с Windows внутрь Linux-контейнера
  volumes {
    host_path      = "C:/Users/spant/Projects"   # Папка на вашей Windows
    container_path = "/home/coder/projects"     # Как она видна в контейнере
  }
}
```

Результат: Когда вы открываете VS Code через Coder, вы нажимаете `File -> Open Folder`, переходите в `/home/coder/projects` и сразу видите все ваши локальные файлы с Windows. Все изменения синхронизируются мгновенно в обе стороны.

Раздел 8. Подключение AI-моделей в Terraform-шаблонах

Что такое `host.docker.internal`?

Контейнеры живут в своей изолированной виртуальной сети. Внутри контейнера воркспейса адрес `localhost` указывает **на сам контейнер воркспейса**, а не на ваш ПК с Windows! Поэтому, чтобы контейнер мог «достучаться» до программ, запущенных на вашем физическом компьютере (например, Ollama или LM Studio), Docker предоставляет специальный DNS-адрес:

> **`host.docker.internal`** — это специальный псевдоним Docker, который внутри любого контейнера превращается в реальный IP-адрес вашей хост-машины (Windows).

Пример шаблона Terraform (`main.tf`) с настройкой трех вариантов AI:

```
terraform {
  required_providers {
    coder = {
      source = "coder/coder"
    }
    docker = {
      source = "kreuzwerker/docker"
    }
  }
}

resource "coder_agent" "main" {
  arch           = "amd64"
  os             = "linux"
  startup_script = "echo 'Воркспейс с готовой конфигурацией AI развернут!'"

  # Пробрасываем переменные окружения для плагинов (Roo Code, Continue, Aider и др.)
  env = {
```

```
    # 1. Локальная модель через Ollama (работает на Windows)
    OLLAMA_HOST = "http://host.docker.internal:11434"

    # 2. Локальная модель через LM Studio (работает на Windows)
    LM_STUDIO_HOST = "http://host.docker.internal:1234/v1"

    # 3. Внешние бесплатные/платные модели через OpenRouter API
    OPENROUTER_API_KEY = "sk-or-v1-your-openrouter-key-here"
    OPENROUTER_BASE_URL = "https://openrouter.ai/api/v1"
    DEFAULT_AI_MODEL   = "meta-llama/llama-3.3-70b-instruct:free" # Бесплатная модель OpenRouter
  }
}

resource "docker_container" "workspace" {
  count = data.coder_workspace.me.start_count
  image = "codercom/example-base:ubuntu"
  name  = "coder-${data.coder_workspace.me.owner}-${data.coder_workspace.me.name}"

  # Обязательно разрешаем контейнеру домен host.docker.internal в Linux
  extra_hosts {
    host = "host.docker.internal"
    ip   = "host-gateway"
  }

  entrypoint = ["sh", "-c", coder_agent.main.init_script]

  env = [
    "CODER_AGENT_TOKEN=${coder_agent.main.token}"
  ]
}

data "coder_workspace" "me" {}
```

Раздел 9. Автоматическое управление SSH-ключами

Вам **не нужно вручную создавать, копировать или передавать SSH-ключи** между Windows и контейнерами.

1. Платформа **Coder берет управление SSH-ключами на себя**.
2. При клике на кнопку **VS Code Desktop** браузер передает секретный токен локальному клиенту Coder CLI на Windows.
3. CLI Coder автоматически генерирует временную SSH-конфигурацию в `C:\Users\spant\.ssh\config`.
4. Расширение `Remote - SSH` в VS Code прозрачно подключается к нужному воркспейсу через созданный защищенный туннель.

Раздел 10. Схема механики сокета (`/var/run/docker.sock`)



1. **Проблема:** Контейнер Coder изолирован. Он не имеет прямого доступа к ресурсам ОС Windows.
2. **Решение (Сокет):** Проброс `- /var/run/docker.sock:/var/run/docker.sock` дает контейнеру Coder доступ к «командному пункту» Docker Desktop.
3. **Механизм работы:**
   - Сервер Coder пишет приказ создать контейнер в проброшенный файл-сокет.
   - Демон Docker Desktop на хост-машине считывает команду из сокета.
   - Docker Desktop создает и запускает дочерний Linux-контейнер воркспейса на хост-машине.